

CAPSTONE PROJECT REPORT

Selenium Web Automation Framework

TutorialsNinja Demo Application

Student Name	Shania Owais
Enrollment No.	12023002029073
Testing Framework	Selenium + Pytest
Application Under Test	TutorialsNinja Demo

Project Type: Functional UI Automation

Table of Contents

Table of Contents	2
1. Introduction	3
2. Project Objectives	3
3. Application Under Test	3
4. Technologies and Tools	3
5. Project Structure	3
6. Page Object Model.....	4
6.1 LoginPage	4
6.2 HomePage	4
6.3 ProductPage	4
6.4 CartPage	4
7. Utilities and Configuration	4
7.1 config.ini and config_reader.py.....	4
7.2 csv_reader.py	4
7.3 screenshot.py	5
8. Test Cases	5
9. Pytest Fixture and Execution Flow	5
10. Pytest Configuration	5
11. HTML Reporting	5
12. Unittest Implementation	6
13. Demonstration Flow	6
14. Results	6
15. Advantages.....	6
16. Limitations and Future Enhancements	7
17. Conclusion	7
Appendix – Key Commands	7

1. Introduction

This project is a Selenium-based web automation framework developed to automate functional test scenarios for the TutorialsNinja Demo e-commerce application. The framework uses the Page Object Model (POM), Pytest, reusable utility modules, external test data, configuration management, screenshots, and HTML reporting.

The main automated workflows are user login, product search, and adding a product to the shopping cart. A separate Python unittest implementation is also included for the login workflow.

2. Project Objectives

- Automate important functional workflows using Selenium WebDriver.
- Implement Page Object Model for reusable and maintainable page actions.
- Use Pytest as the primary test execution framework.
- Use CSV files for external product and login test data.
- Use config.ini for centralized browser and application configuration.
- Capture screenshots when a test fails.
- Generate an HTML execution report using pytest-html.
- Demonstrate the login flow using Python unittest.

3. Application Under Test

The application under test is the TutorialsNinja Demo e-commerce website. Chrome is used as the browser. The automated areas are My Account/Login, Product Search, Product Details, Add to Cart, and Shopping Cart verification.

4. Technologies and Tools

Technology / Tool	Purpose	Project Usage
Python	Programming language	Tests, POM classes and utilities
Selenium WebDriver	Browser automation	Chrome browser interaction
Pytest	Test framework	Primary test execution
pytest-html	Reporting	HTML execution report
pytest-metadata	Report metadata	Project/environment metadata
unittest	Python test framework	Separate login test
CSV	External test data	Product and login data
configparser	Configuration	Reads config.ini

5. Project Structure

```
Capstone_SeleniumAutomation/
├── config/config.ini
├── data/test_data.csv
├── pages/
│   ├── home_page.py
│   ├── login_page.py
│   ├── product_page.py
│   └── cart_page.py
├── tests/
│   ├── test_login.py
│   ├── test_search.py
│   └── test_cart.py
├── unittest_tests/test_login_unittest.py
├── utilities/
│   ├── config_reader.py
│   ├── csv_reader.py
│   └── screenshot.py
├── reports/
├── screenshots/
├── conftest.py
├── pytest.ini
└── requirements.txt
```

6. Page Object Model

6.1 LoginPage

Contains locators and methods for the email field, password field and Login button. Methods include `enter_email()`, `enter_password()`, and `click_login()`.

6.2 HomePage

Contains the search box and search button. It provides `search_product()` and `is_product_displayed()` methods.

6.3 ProductPage

Contains the Add to Cart button and Shopping Cart link. It provides `click_add_to_cart()` and `go_to_cart()`.

6.4 CartPage

Verifies that the selected product is displayed in the shopping cart.

7. Utilities and Configuration

7.1 config.ini and config_reader.py

`config.ini` stores reusable settings such as browser and application URL. `config_reader.py` uses `configparser` to read the `DEFAULT` section. This keeps environment settings out of the test logic.

7.2 csv_reader.py

The CSV reader uses `csv.DictReader` to load rows as dictionaries. The tests retrieve values by column names such as `product`, `email`, and `password`.

7.3 screenshot.py

The screenshot utility creates the screenshots directory when required, adds a timestamp to the filename, saves the browser screenshot, and returns its path.

8. Test Cases

ID	Test Case	Main Flow	Expected Result	Status
TC-01	Login	Open site → My Account → Login → credentials → submit	Login verification succeeds	PASS
TC-02	Product Search	Read product → search → verify	Requested product displayed	PASS
TC-03	Add to Cart	Search → open product → add → cart → verify	Requested product present in cart	PASS
TC-04	Unittest Login	Same login workflow using unittest	Login test passes	PASS

9. Pytest Fixture and Execution Flow

The driver fixture in `conftest.py` reads the configuration, creates a Chrome WebDriver, maximizes the window, yields the driver to the test, and quits the browser during teardown.

1. Pytest discovers tests using `pytest.ini`.
2. The driver fixture starts Chrome.
3. The test reads configuration and/or CSV data.
4. Page objects perform application actions.
5. Assertions verify expected behavior.
6. Failure handling captures a screenshot.
7. The fixture closes the browser.
8. `pytest-html` can generate the execution report.

10. Pytest Configuration

`pytest.ini` configures test discovery using the tests directory and standard `test_file/function` naming.

```
[pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
```

11. HTML Reporting

`pytest-html` is used to generate an HTML summary of the automated execution. `pytest-metadata` is used for project, application, browser, environment, and test-type information.

```
pytest -v --html=reports/report.html
```

12. Unittest Implementation

A separate unittest test class demonstrates the login flow. setUp() launches Chrome, the test method performs the login and verifies the result, and tearDown() closes the browser.

```
python -m unittest unittest_tests.test_login_unittest
```

13. Demonstration Flow

1. Introduce the project and objective.
2. Show the folder structure.
3. Explain config.ini and CSV test data.
4. Explain the Page Object Model classes.
5. Explain the utility modules and conftest.py.
6. Run the Pytest suite and demonstrate the browser flow.
7. Show the successful Pytest results.
8. Run the unittest login test.
9. Show the HTML report and screenshots.

14. Results

The implemented Pytest suite contains three functional tests: login, product search, and add product to cart. The final execution successfully completed all three. The separate unittest login test also completed successfully.

Execution	Tests	Result
Pytest suite	3	All Passed
Pytest Login	1	Passed
Pytest Search	1	Passed
Pytest Cart	1	Passed
Unittest Login	1	Passed

15. Advantages

- Reusable page objects reduce duplication.
- External CSV data makes test data easy to update.
- Centralized configuration simplifies changes.
- Screenshot capture supports failure analysis.

- Pytest fixtures provide common browser setup and teardown.
- HTML reporting provides an execution summary.
- The framework remains simple and easy to explain.

16. Limitations and Future Enhancements

The framework is intentionally kept simple for the capstone scope. Possible future improvements include:

- Replace fixed `time.sleep()` calls with explicit waits where appropriate.
- Add positive and negative test scenarios.
- Parameterize tests for multiple products and credentials.
- Add cross-browser execution.
- Enhance step-level reporting and attachments.
- Integrate the suite with CI/CD.

17. Conclusion

The Capstone Selenium Automation project demonstrates a structured approach to web UI automation using Python and Selenium WebDriver. It combines Page Object Model design, Pytest, external CSV data, centralized configuration, reusable utilities, screenshots, HTML reporting, and a separate unittest implementation.

The separation of tests, page objects, data, configuration, and utilities makes the framework readable, reusable, and maintainable while satisfying the core objectives of the capstone project.

Appendix – Key Commands

```
venv\Scripts\activate
pip install -r requirements.txt
pytest -v
pytest -v --html=reports/report.html
python -m unittest unittest_tests.test_login_unittest
```

End of Report